

Joseph Cassello Jr

CS 499 Enhancement Two: Algorithms and Data Structures

Professor Phillips

February 1, 2026

Artifact Description:

The artifact selected for this enhancement is a C++ application originally developed in CS 300, Analysis and Design. Its primary function is to manage and search through a large dataset of eBid auction bids using a Binary Search Tree (BST) structure. The original program allowed for basic insertion, deletion, and searching of bid IDs. While functional, the original code relied on a standard BST, which lacked self balancing capabilities and robust memory management.

Justification for Inclusion:

I selected this artifact for my ePortfolio because it represents a fundamental challenge in Computer Science: ensuring consistent performance as datasets scale. By enhancing this specific project, I have demonstrated my ability to implement complex, self balancing logic that is used in real world database engines and file systems. The specific components that showcase my skills include algorithmic optimization where I transitioned the data structure to an AVL tree. I also demonstrated complexity management by implementing the four standard rotation cases known as Left-Left, Right-Right, Left-Right and Right-Left to maintain a balance factor between -1 and 1 for every node. Finally, I improved memory safety by implementing a recursive destructor to properly deallocate memory and prevent potential leaks.

Alignment with Course Outcomes:

I have successfully met the course outcomes I targeted in Module One. For Course Outcome 3, I demonstrated the ability to evaluate computing solutions and manage design trade offs by choosing an AVL tree over a standard BST. This involved trading a small amount of insertion complexity for a guaranteed $O(\log n)$ search time, ensuring the system remains efficient even if the input data is sorted. Regarding Course Outcome 4, the use of recursive height balancing and rotation algorithms illustrates my ability to use well founded and innovative techniques to deliver professional grade software solutions.

Reflection on the Process:

Enhancing this artifact was a significant learning experience that moved me beyond basic data structure implementation into advanced algorithm engineering. The primary challenge I faced was the `removeNode` function. Unlike a simple BST removal, an AVL removal requires unwinding the recursion to check the balance factor of every ancestor node and performing rotations where necessary. This required a deep dive into pointer manipulation and recursion logic. Through this process, I learned that working code is not always quality code. My IT background has taught me to fix what is broken, but this project taught me to anticipate performance bottlenecks before they happen. I had to debug several edge cases where the tree would lose its balance factor after multiple deletions, which strengthened my testing and debugging skills. This project has given me the confidence to handle the complex logic required in systems architecture and back end development.